

MODULE 1

Python Programming — Answers

Introduction, Data Types, Decision Control and Loop Control

Document: M1_22-07-2026_ans

PART A — Short Answer Questions (3 Marks)

1. Explain the key features of Python programming language.

Python is a high-level, general-purpose programming language known for its simple, readable syntax. Its key features are:

- Easy to learn and read — syntax is close to plain English, no curly braces or semicolons needed.
- Interpreted language — code runs line by line through the Python Virtual Machine, so there is no separate manual compilation step for the programmer.
- Dynamically typed — a variable's data type is decided automatically at run time; no need to declare types.
- Free and open source — Python and most of its libraries can be downloaded and used at no cost.
- Portable / platform independent — the same source code runs on Windows, Linux and macOS without change.
- Large standard library — ready-made modules for file handling, math, networking, data science, GUI, etc.
- Object-oriented — supports classes, objects, inheritance and other OOP concepts.
- Extensible and embeddable — can be integrated with C, C++, Java code, and embedded inside other applications.

2. Discuss different data types in Python along with their uses.

A data type tells the interpreter what kind of value a variable holds and what operations can be performed on it. Python's built-in data types are:

Data Type	Example	Use
int	age = 21	Store whole numbers (counting, indexing, marks).
float	price = 99.50	Store decimal / real numbers (money, measurements).
str	name = "Riya"	Store text (names, messages, file paths).
bool	is_valid = True	Store True/False for decisions and flags.
list	marks = [45, 67, 89]	Ordered, changeable collection of items.
tuple	point = (2, 3)	Ordered, unchangeable collection — fixed records.
dict	student = {"name": "A", "age": 20}	Store data as key–value pairs (records/lookup tables).
set	roll_nos = {1, 2, 3}	Store unique, unordered items — remove duplicates fast.

Python decides the type automatically from the value assigned, and the `type()` function can be used to check it, e.g. `type(21)` gives `<class 'int'>`.

3. Explain the structure of if, if-else, and if-elif-else along with their functionalities.

These are decision-making (conditional) statements. They let a program choose which block of code to run based on whether a condition is True or False.

if statement

```
if condition:
    statement(s)    # runs only if condition is True
```

Executes a block only when the condition is true; otherwise the block is simply skipped.

if-else statement

```
if condition:
    statement(s)    # runs if condition is True
else:
    statement(s)    # runs if condition is False
```

Provides two paths — exactly one of the two blocks always executes.

if-elif-else statement

```
if condition1:
    statement(s)
elif condition2:
    statement(s)
elif condition3:
    statement(s)
else:
    statement(s)    # runs if none of the above are True
```

Used when there are more than two possible outcomes; conditions are tested in order, and the first True one runs, after which the rest are skipped.

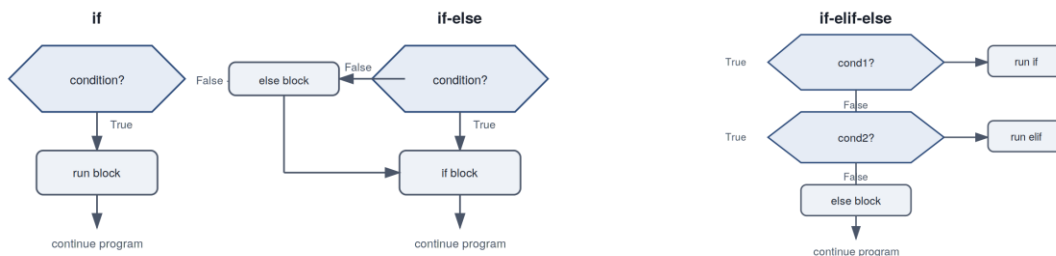


Figure: Flow of if, if-else, and if-elif-else

Example:

```
marks = 72
if marks >= 90:
    print("Grade A")
elif marks >= 60:
    print("Grade B")
else:
    print("Grade C")
# Output: Grade B
```

4. Explain the use of break and continue statements in Python loops.

break

Immediately stops (terminates) the loop it is in, and control jumps to the statement right after the loop, no matter how many iterations were left.

```
for i in range(1, 10):  
    if i == 5:  
        break  
    print(i)  
# Output: 1 2 3 4
```

continue

Skips the remaining statements of the current iteration only, and moves on to the next iteration of the loop; the loop itself keeps running.

```
for i in range(1, 6):  
    if i == 3:  
        continue  
    print(i)  
# Output: 1 2 4 5
```

PART B — Long Answer Questions (5 Marks)

1. Write a Python program and explain its working procedure.

Program: find whether a number is even or odd.

```
num = int(input("Enter a number: "))

if num % 2 == 0:
    print(num, "is Even")
else:
    print(num, "is Odd")
```

Working: input() reads a value typed by the user as text, int() converts that text to a whole number and stores it in num. The expression num % 2 computes the remainder when num is divided by 2 — this is 0 for even numbers. The if-else statement then prints the correct message.

How the program actually runs — from IDE to output

A Python program does not go straight from readable text to an answer. It passes through several stages every time it is run:

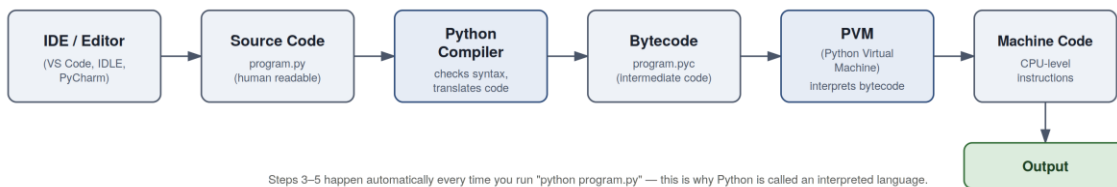


Figure: Execution pipeline of a Python program

- IDE / Editor — the program is typed and saved as a .py file, e.g. even_odd.py, using an editor like IDLE, VS Code or PyCharm.
- Source Code — the plain text file containing Python statements, written in human-readable form.
- Python Compiler — when the program is run, Python's compiler first checks the code for syntax errors and translates it into an intermediate, low-level form.
- Bytecode (.pyc) — this intermediate output (instructions such as LOAD_FAST, BINARY_MODULO) is not machine code yet; it is portable across operating systems.
- PVM (Python Virtual Machine) — the interpreter engine that reads the bytecode instruction by instruction and executes it; this is the actual "Python interpreter" people refer to.
- Machine Code — as the PVM executes each bytecode instruction, it is ultimately carried out as CPU-level machine instructions on the underlying hardware.
- Output — the result (here, "Even" or "Odd") is displayed back to the user, e.g. through the console.

Note: Steps 3–6 happen automatically and instantly each time "python even_odd.py" is run. Because compiling to bytecode and executing happen together behind the scenes, Python is called an interpreted language, even though a compilation step does take place internally.

2. Explain the features and advantages of Python.

Python's core features were listed in Part A, Q1. Beyond those features, its practical advantages are:

- Rapid development — fewer lines of code are needed compared to languages like C++ or Java, so programs can be written and tested quickly.
- Huge ecosystem — libraries such as NumPy, Pandas, Django, Flask and TensorFlow cover data science, web development and AI, avoiding the need to write everything from scratch.
- Strong community support — being open source and widely used means documentation, tutorials and forum help are easy to find.
- Good for beginners as well as professionals — the same simple syntax scales up to large, complex, real-world systems.
- Cross-platform — Python installed on any OS can run the same .py file without modification.
- Integration friendly — Python code can call functions written in C/C++ for speed, or be embedded inside other applications.

Together, these make Python popular for automation scripting, web back-ends, data analysis, machine learning and teaching programming concepts.

3. Describe the differentiate selection/decision operators in Python and describe their uses.

Selection (decision) statements let a program choose between alternative actions based on a condition. Python provides the following forms:

Statement	Number of paths	Typical use
if	One (runs or skipped)	Simple checks, e.g. validating a single condition before proceeding.
if-else	Two (either / or)	Binary decisions, e.g. pass/fail, eligible/not eligible.
if-elif-else	Many, evaluated in order	Grading systems, menu choices, multiple category checks.
Nested if	One condition inside another	Checking a sub-condition only when an outer condition is already True, e.g. checking discount only if a customer is a member.
match-case (Python 3.10+)	Multiple fixed patterns	Cleaner alternative to long if-elif chains when comparing one variable against many fixed values.

Example of a nested if:

```
age = 20
has_id = True

if age >= 18:
    if has_id:
        print("Entry allowed")
    else:
        print("ID required")
else:
    print("Not eligible")
```

Choosing the right form keeps code readable: use if for a single check, if-else for two outcomes, and if-elif-else (or match-case) when there are several mutually exclusive cases.

4. Explain the applications of for and while loops in Python and relate them to real-life situations.

for loop

Used when the number of repetitions is known in advance or when iterating over a fixed sequence (a list, string, range, etc.).

```
for day in ["Mon", "Tue", "Wed", "Thu", "Fri"]:  
    print("Attend college on", day)
```

Real-life analogy: going through a fixed weekly timetable, day by day, from Monday to Friday — the number of days is already fixed.

while loop

Used when repetition should continue only as long as a condition remains true, and the exact number of repetitions is not known beforehand.

```
balance = 500  
while balance > 0:  
    balance -= 100  
    print("Remaining balance:", balance)
```

Real-life analogy: withdrawing money from an account again and again until the balance runs out — you keep going only while money remains, not for a fixed number of times.

Aspect	for loop	while loop
Best used when	Number of iterations is known	Repetition depends on a condition
Typical source	Iterates over range()/list/string	Iterates until a condition becomes False
Real-life example	Marking attendance for each of 30 students	Keep asking for a password until it is correct

5. Explain the use of break, continue, and pass statements and represent their rules in loop control.

These three statements alter the normal top-to-bottom flow inside a loop:

Statement	Effect	Rule
break	Ends the loop immediately	Used to stop looping as soon as a target/condition is met, e.g. stop searching once an item is found.
continue	Skips the rest of the current iteration	Used to bypass certain values while still continuing the loop, e.g. skip negative numbers in a sum.
pass	Does nothing at all	Used as a placeholder where a statement is syntactically required but no action is needed yet, e.g. an empty function body during planning.

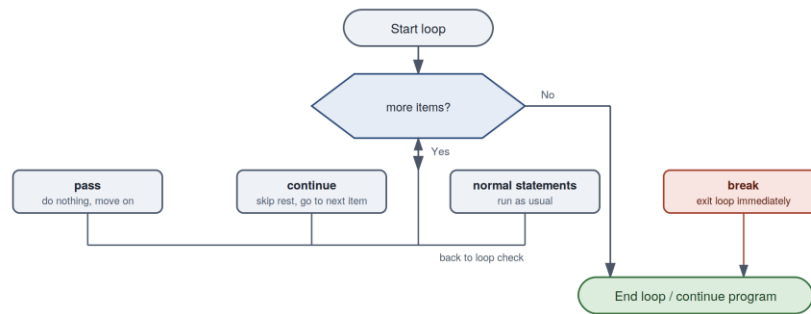


Figure: How break, continue and pass affect loop flow

Example combining all three:

```
for num in range(1, 8):
    if num == 3:
        continue      # skip printing 3, go to next number
    if num == 6:
        break         # stop the loop completely at 6
    if num == 4:
        pass          # placeholder, does nothing special
    print(num)
# Output: 1 2 4 5
```

break and continue always act on the innermost loop they are written in; pass never affects loop flow at all — it is only used to satisfy Python's syntax when a block cannot be left empty.